Jacob Walcutt

jacobwalcutt.dev | jacobwalcutt@gmail.com | linkedin.com/in/jacob-walcutt | github.com/jwalcutt

Education

Cleveland State University — Cleveland, OH

Bachelor of Science in Computer Science | GPA: 3.90/4.0 — *Jan. 2023 – May 2026*

- Relevant Courses: Quantum Machine Learning, Artificial Intelligence, Software Engineering, Computer Security, Computer Architectures, Database Concepts, Operating Systems, Computer Networks, Systems Programming

Experience

Software Engineering Co-op — May 2025 – Dec. 2025

nVent — *Solon, OH*

- Designed and trained a 1D Convolutional Autoencoder in **PyTorch** on sliding 512-sample windows of field-collected waveforms, flagging arc faults as reconstruction-error spikes that matched ground-truth current readings
- Engineered a **Python** scoring pipeline that clustered flagged segments into discrete arc events and quantified their magnitude against the reconstructed wave, nearly eliminating all false positives
- Built an interactive **Plotly Dash** labeling tool annotating arc events into a JSON ground-truth schema, powering an **Optuna** Bayesian search of 10+ hyperparameters against an F1-score objective
- Automated a Raspberry Pi weld-testing station with a **Node.js** (Express) API firing relay hardware over GPIO and logging outcomes server-side, then parsed the logs to visualize pass/fail rates by part type
- Extended an **Excel/VBA** Jira automation tool tracking 1,000+ projects, estimated to save $100k+ annually

Software Development Intern — June 2024 – Dec. 2024

VN Services — *Chesterland, OH*

- Built a full-stack database-hosting platform with **Flask** and **SQLite** spanning 30+ REST endpoints, converting raw CSVs into queryable web databases via **pandas** ingestion with automatic encoding detection
- Engineered a schema-agnostic query engine with parameterized SQL, delivering filtering, range queries, aggregations, pagination, and CSV/Excel export over tables of 380,000+ rows
- Secured the platform with role-based access control and bcrypt password hashing, adding per-user database permissions and server-side session revocation for instant account bans
- Containerized the application with **Docker** and deployed it to **AWS Fargate (ECS)** via Amazon ECR, configuring task definitions and networking to serve the platform from the cloud
- Extended the **Flask** backend of an internal training platform to serve dynamic course content, and automated a **Python** data-sync workflow that fed database updates into reporting visualizations

Projects

Engram: Cross-Modal Memory Engine | *RAG, Embeddings, Ollama, FastAPI, OCR*

- Implemented a local-first ingestion pipeline in **Python** and **SQLite** unifying four modalities, indexing 24,000+ evidence chunks across 450+ sources into one queryable store
- Enabled semantic search with fastembed ONNX embeddings in **SQLite** scored by cosine similarity, deliberately rejecting an ANN index at 24k-chunk scale to hold sub-100ms retrieval
- Designed a four-level hierarchical memory model using local LLM extraction, temporal session grouping, and embedding clustering, deriving 3,477 nodes and 148,000 edges that expose cross-project themes
- Shipped a full interface layer with **FastAPI** and **React**, delivering a streaming multi-turn chat console with conversation persistence and per-message claim-to-evidence drill-down backed by 527 automated tests

Improving LLM Reasoning through Autoformalization | *LLMs, Lean 4, RAG, Langchain, Tool Use*

- Built an agentic theorem-formalization system in **Python** and **LangChain** on a 4-person team, equipping LLMs with a Mathlib search tool (Loogle) and a JSON-RPC Lean 4 compiler interface for iterative self-correction
- Developed a full-stack AI chat application in **React** and **TypeScript** over an async **Flask** API (Quart), streaming LLM tokens over HTTP into live-rendered LaTeX, Lean syntax highlighting, and reasoning-model thinking blocks
- Constructed a FAISS vector index of 125,000 Mathlib4 code chunks with OpenAI embeddings, delivering top-5 semantic retrieval of formal definitions in under 200 ms
- Built a semantic-consistency benchmark in **Python** that informalizes 186 Lean theorems to natural language and scores cosine similarity, quantifying formalization stability with 10 pairwise similarity scores per theorem

Technical Skills

Languages: Python, JavaScript, TypeScript, C, C#, C++, Rust, Java, HTML, CSS, SQL
Libraries & Frameworks: PyTorch, Flask, Pandas, FastAPI, LangChain, NumPy, React, Optuna, Node.js, Next.js
Developer Tools: SQLite, PostgreSQL, Docker, AWS Fargate, AWS ECS, Terraform, Supabase, Microsoft SQL Server